

关系数据理论的主要内容是关系规范化理论,用来指导数据库逻辑设计,具体包括两部分:一是给出判断数据库逻辑设计“好坏程度”的准则;二是如果逻辑设计中存在“不好”的关系模式,如何将其修改为“好”的关系模式。

### 本章导读

在关系数据库模式设计中,我们面临两个问题,第一,什么是一个“好”的关系数据库模式,其标准是什么。如果没有判断的标准,就很难对一个具体的设计进行评价。第二,能否开发数据库设计辅助工具来减轻设计者的负担。本章6.1和6.2节介绍关系规范化理论,用于回答第一个问题;6.3和6.4节介绍函数依赖公理系统以及模式分解算法,探索自动进行模式分解的方法,用于回答第二个问题。

本章首先从数据库逻辑设计中如何构造一个好的数据库模式这一问题出发,阐明关系规范化理论研究的实际背景,然后讨论两类最重要的数据依赖——函数依赖和多值依赖。读者要牢固掌握数据依赖的基本概念,如函数依赖、部分函数依赖、完全函数依赖、传递函数依赖等,掌握1NF到4NF的概念、规范化的含义和作用;能够了解各种范式以及在插入、删除和更新数据时可能出现的问题,并直观地描述解决办法。

本章进一步讨论函数依赖的推理规则,介绍函数依赖公理系统,讲解保持函数依赖的模式等价准则及相应的模式分解算法。要掌握这些算法并在数据库模式设计中应用。6.5节是选读内容,介绍无损连接的模式分解概念及相应算法,并解释其与保持函数依赖的模式分解之间的关系。

读者学习本章后应能够根据应用语义完整地写出关系模式的数据依赖集合,并能根据数据依赖分析某一个关系模式属于第几范式,能够运用介绍的有关算法辅助设计和优化数据库模式。

## 6.1 问题的提出

前面已经讨论了数据库系统的一般概念,介绍了关系数据库的基本概念、关系模型的三要

素以及关系数据库的标准语言 SQL。但是还有一个很基本的问题尚未涉及：针对一个具体问题，应该如何构造一个适合它的数据库模式，即应该构造几个关系模式，每个关系模式由哪些属性组成等。如何判断这个模式是好的，也就是设计的标准如何，是关系数据库逻辑设计要解决的问题。

首先回顾一下关系模式的形式化定义。

在第2章关系数据库中已经讲过，一个关系模式可以表示为

$$R(U, D, \text{DOM}, F)$$

这里：

- ① 关系名  $R$  是符号化的元组语义。
- ②  $U$  为组成该关系模式的属性名集合。
- ③  $D$  为  $U$  中的属性所来自的域。
- ④  $\text{DOM}$  为属性向域的映射。
- ⑤  $F$  为  $U$  上的一组数据依赖。

由于  $D$ 、 $\text{DOM}$  与模式设计关系不大，因此在本章中把关系模式表示为  $R(U, F)$ 。当且仅当  $U$  上的一个关系  $r$  满足  $F$  时，称  $r$  为关系模式  $R(U, F)$  的一个关系。

作为二维表，关系要符合一个最基本的条件是每一个分量必须是不可分的数据项。满足了这个条件的关系模式就属于第一范式(1NF)。

在模式设计中，给定一个模式  $S_\phi$ ，它仅由单个关系模式组成，问题是要设计一个模式  $S_D$ ，它与  $S_\phi$  等价(等价的确切含义后面再展开)，但在某些指定的方面更好一些。这里通过一个例子来说明一个不好的模式会有些什么问题，分析它们产生的原因，并从中找出设计一个好的关系模式的办法。

在举例之前，先非形式地讨论一下数据依赖的概念。

数据依赖是一个关系属性与属性之间的一种约束关系。这种约束关系是通过属性间值的相等与否体现出来的数据间的相关联系。它是现实世界属性间相互联系的抽象，是数据内在的性质和语义的体现。

人们已经提出了许多种类型的数据依赖，其中最重要的是函数依赖(functional dependency, FD)和多值依赖(multivalued dependency, MVD)。

函数依赖极为普遍地存在于现实生活中。比如描述一个学生的关系，可以有学号(Sno)、姓名(Sname)、所在学院(School)等属性。由于通常一个学号只对应一个学生，一个学生只能在一个学院学习，因而当“学号”值确定之后，学生的姓名及其他属性值也就被唯一地确定了。属性间的这种依赖关系类似于数学中的函数  $y=f(x)$ ，自变量  $x$  确定之后，相应的函数值  $y$  也就唯一地确定了。

类似的有  $\text{Sname}=f(\text{Sno})$ ， $\text{School}=f(\text{Sno})$ ，即  $\text{Sno}$  函数决定  $\text{Sname}$ ， $\text{Sno}$  函数决定  $\text{School}$ ，或者说  $\text{Sname}$  和  $\text{School}$  函数依赖于  $\text{Sno}$ ，记作  $\text{Sno} \rightarrow \text{Sname}$ ， $\text{Sno} \rightarrow \text{School}$ 。

[例 6.1] 建立一个描述学校教务的数据库，该数据库涉及的对象包括学生的学号(Sno)、

所在学院(School)、所在学院院长姓名(Mname)、选修课的课程号(Cno)和成绩(Grade)。假设用一个单一的关系模式 Student 来表示,则该关系模式 Student( $U, F$ )的属性组为

$$U = \{ \text{Sno}, \text{School}, \text{Mname}, \text{Cno}, \text{Grade} \}$$

现实世界的已知事实(语义)告诉我们:

- ① 一个学院有若干学生,但一个学生只属于一个学院。
- ② 一个学院只有一名(正职)院长。
- ③ 一个学生可以选修多门课程,每门课程有若干学生选修。
- ④ 每个学生学习每一门课程有一个成绩。

于是得到  $U$  上的一组函数依赖  $F$  (如图 6.1 所示)。

$$F = \{ \text{Sno} \rightarrow \text{School}, \text{School} \rightarrow \text{Mname}, (\text{Sno}, \text{Cno}) \rightarrow \text{Grade} \}$$

如果只考虑函数依赖这一种数据依赖,可以得到一个描述学生的关系模式 Student( $U, F$ )。

表 6.1 是某一时刻关系模式 Student 的一个实例,即数据表。

表 6.1 Student 表

| Sno | School | Mname | Cno | Grade |
|-----|--------|-------|-----|-------|
| S1  | 信息学院   | 张明    | C1  | 95    |
| S2  | 信息学院   | 张明    | C1  | 90    |
| S3  | 信息学院   | 张明    | C1  | 88    |
| S4  | 信息学院   | 张明    | C1  | 70    |
| S5  | 信息学院   | 张明    | C1  | 78    |
| ⋮   | ⋮      | ⋮     | ⋮   | ⋮     |

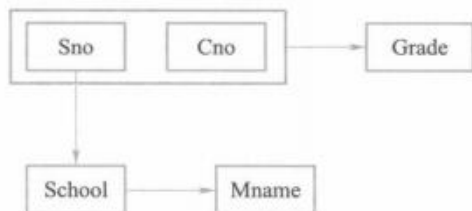


图 6.1 Student 上的一组函数依赖

但是,这个关系模式存在以下问题:

① 数据冗余(data redundancy)。比如,院长姓名重复出现,重复次数与该学院所有学生的所有课程成绩出现次数相同,如表 6.1 所示。这将浪费大量的存储空间。

② 更新异常(update anomaly)。由于数据冗余,当更新数据库中的数据时,系统要付出很大的代价来维护数据库的完整性,否则会面临数据不一致的危险。比如,某学院更换院长后,必须修改与该学院学生有关的每一个元组。

③ 插入异常(insertion anomaly)。如果一个学院刚成立,尚无学生,则无法把这个学院及其院长的信息存入数据库。

④ 删除异常(deletion anomaly)。如果某个学院的学生全部毕业了,则在删除该学院学生信息的同时,这个学院及其院长的信息也丢掉了。

鉴于存在以上种种问题,可以得出这样的结论:例 6.1 的 Student 关系模式不是一个好的模式。一个好的模式应当不会发生插入异常、删除异常和更新异常,数据冗余应尽可能

能少。

为什么会发生这些问题呢？

这是因为这个模式中的函数依赖存在某些不好的性质。这正是本章要讨论的问题。假如把这个单一的模式改造一下，分成以下三个关系模式：

```
S( { Sno, School } , { Sno → School } );
SC( { Sno, Cno, Grade } , { ( Sno, Cno ) → Grade } );
SCHOOL( { School, Mname } , { School → Mname } )
```

则这三个模式都不会发生插入异常、删除异常和更新异常的问题，数据的冗余也得到了控制。

一个模式的数据依赖会有哪些不好的性质，如何改造一个不好的模式，这就是规范化要讨论的内容。

## 6.2 规范化

本节首先讨论一个关系的属性间不同的依赖情况，以及如何根据属性间依赖情况判定关系是否具有某些不合适的性质。通常按属性间依赖情况区分关系的规范化程度为第一范式、第二范式、第三范式和第四范式等。然后直观地描述如何将具有不合适性质的关系转换为更合适的形式。

### 6.2.1 函数依赖

**定义 6.1** 给定关系模式  $R(U, F)$ ， $X, Y$  是  $U$  的子集。若对于  $R(U, F)$  的任意一个可能的关系  $r$ ， $r$  中不可能存在两个元组在  $X$  上的属性值相等，而在  $Y$  上的属性值不等，则称  $X$  函数确定  $Y$  或  $Y$  函数依赖于  $X$ ，记作  $X \rightarrow Y$ 。

函数依赖和别的数据依赖一样是语义范畴的概念，只能根据语义来确定一个函数依赖。例如，“姓名  $\rightarrow$  年龄”这个函数依赖只有在该部门没有同名人的条件下成立。如果允许有同名人，则年龄就不再函数依赖于姓名了。

设计者也可以对现实世界做强制性规定，例如规定不允许同名人出现，因而使“姓名  $\rightarrow$  年龄”函数依赖成立。这样当插入某个元组时该元组上的属性值必须满足规定的函数依赖，若发现有同名人存在，则拒绝插入该元组。

**注意：**函数依赖不是指关系模式  $R$  的某个或某些关系满足的约束条件，而是指  $R$  在不同时刻的关系实例均要满足的约束条件。

例如，6.1 节关系模式  $\text{Student}(U, F)$  中有  $\text{Sno} \rightarrow \text{School}$  成立，也就是说在任何时刻该模式的关系实例（即  $\text{Student}$  数据表）中，不可能存在两个元组在  $\text{Sno}$  上的值相等，而在  $\text{School}$  上的值不等。因此，表 6.2 的  $\text{Student}$  表是错误的。因为表中有两个元组在  $\text{Sno}$  上都等于“S1”，而